OAgents: A Behavioral Envelope Standard for Trustworthy AI Agent Operations

An Implementation Profile of the NIST AI Risk Management Framework (AI RMF 1.0)

JD Longmire
Northrop Grumman Fellow (unaffiliated research)
ORCID: 0009-0009-1383-7698
jdlongmire@outlook.com

Field	Value
Document Type	AI RMF Implementation Profile
AI RMF Reference	NIST AI 100-1 (January 2023)
GenAI Profile Reference	NIST AI 600-1 (July 2024)
Profile Version	1.0
Date	April 2026

Abstract

The deployment of large language models as operational agents -- managing infrastructure, generating enterprise content, executing multi-step workflows -- has outpaced the development of trust mechanisms for their behavior. Current AI agent frameworks optimize for capability (tool calling, chain-of-thought, function execution) but provide no standardized guarantees about operational reliability, output quality, or behavioral consistency.

This document proposes OAgents, a control and conformance profile for operational AI agent trustworthiness, presented as an AI RMF Implementation Profile. The profile defines a behavioral envelope -- a structured set of pre-execution gates, post-execution verification, and operational discipline mechanisms -- and specifies 26 controls across 7 categories with three conformance levels and observable evidence criteria. The standard is model-agnostic: behavioral guarantees are properties of the envelope, not properties of the underlying model.

Presented as an Implementation Profile of the NIST AI Risk Management Framework (AI RMF 1.0, NIST AI 100-1), the OAgents standard maps a 26-component taxonomy to AI RMF functions, categories, and subcategories. A reference implementation demonstrates the architectural feasibility of all 26 components. No generalized empirical benchmarks are claimed; systematic validation across diverse deployment contexts is identified as community work.

The behavioral envelope is to operational AI agents what OAuth was to identity delegation -- a trust layer that makes delegation safe. OAgents defines that layer as a structured, implementable, NIST-aligned standard.

Foreword

This document is an AI RMF Implementation Profile as defined in Section 6 of NIST AI 100-1. Profiles are implementations of the AI RMF functions, categories, and subcategories for a specific setting or application based on the requirements, risk tolerance, and resources of the Framework user.

The OAgents profile addresses a specific and underserved use case: AI agents operating in enterprise operational roles with authority to take consequential actions -- deploying code, managing infrastructure, generating customer-facing content, and responding to incidents. This is a high-autonomy, high-consequence deployment context. To the authors' knowledge, no prior AI RMF implementation profile has addressed this context at the operational control level; the authors invite correction and comparison from the community.

This profile is submitted as a community contribution in response to NIST's AI Agent Standards Initiative (CAISI, February 2026), which identified the need for industry-led technical standards and open protocols to ensure AI agents function securely and interoperably across enterprise environments.

This profile is consistent with, and builds upon, the NIST AI RMF 1.0 (NIST AI 100-1) and the Generative AI Profile (NIST AI 600-1). This profile explicitly addresses the hallucination risk category from NIST AI 600-1 through the Anti-Hallucination component category. Full mapping to all 13 GenAI risk categories is planned for a subsequent revision.

Terminology in this document follows AI RMF 1.0 definitions. RFC 2119 normative language (MUST, SHOULD, MAY) is used in Section 5 (Component Taxonomy) to specify conformance requirements, consistent with established standards practice.

Development methodology. This standard and its reference implementation were developed using a Human-Curated, AI-Enabled (HCAE) methodology. The human architect defined operational standards, curated behavioral corrections, and determined what "good" looks like across the component categories. The AI operator -- a large language model acting as a generalist operations engineer -- implemented, executed, and maintained the behavioral envelope under those standards. The standard itself is a direct artifact of operating under its own principles: the reference implementation is the system that was used to develop it.

1. Introduction

1.1 The Trust Gap in Enterprise AI Agent Operations

Enterprise organizations are deploying AI agents in progressively higher-stakes operational roles: writing and deploying code, managing cloud infrastructure, generating customer-facing documents, and responding to operational incidents. The underlying models are increasingly capable. The problem is not capability -- it is trust.

When a human operator deploys code to production, the organization trusts that operator because of training, experience, accountability, and process enforcement (code review, CI/CD gates, change management). When an AI agent performs the same action, none of these trust mechanisms exist by default. The agent has no persistent memory of past mistakes. No independent reviewer checks its work. No process gate prevents it from skipping quality checks under prompt pressure. No institutional knowledge carries from one session to the next.

This gap -- between what AI agents can do and what organizations can trust them to do -- is the central barrier to enterprise AI agent adoption. It is empirically documented: the 2025 AI Agent Index found that of 13 deployed agents exhibiting frontier levels of autonomy, only 4 disclosed any agentic safety evaluations, and 25 out of 30 disclosed no internal safety results whatsoever (Casper et al., 2026).

The standards infrastructure to address this gap is nascent. Identity delegation has OAuth (RFC 6749) and OpenID Connect. Network trust has NIST SP 800-207 (Zero Trust Architecture). Software supply chain integrity has SLSA and SBOM frameworks. AI agent behavioral trust has no widely adopted equivalent. Federal agencies operating under Executive Order 14110 on Safe, Secure, and Trustworthy Artificial Intelligence are required to ensure AI systems meet trustworthiness standards -- but operational implementation standards specifying what trustworthy AI agent behavior looks like at the control level are still emerging. OAgents is proposed as a contribution to that developing landscape.

1.2 The OAuth Parallel

In the early 2000s, web applications faced an analogous trust problem. Users needed to grant third-party applications access to their data on other platforms, but the only mechanism was sharing passwords -- unverifiable, unscoped, irrevocable. OAuth solved this by introducing a standardized trust layer: tokens that carry scoped permissions, issued by identity providers, verifiable by any relying party.

OAuth did not replace applications. It wrapped them in a trust mechanism that made delegation safe.

AI agents face the same structural problem. Organizations need to delegate operational authority to AI agents, but the only mechanism is prompt instructions -- unverifiable, unenforceable, and forgotten when context compresses. The industry needs a standardized trust layer for AI agent operations: a mechanism that makes delegation safe regardless of which model powers the agent.

The analogy has limits worth acknowledging. OAuth solved a sharply bounded authorization problem with a well-defined token exchange protocol. AI agent behavioral trust is a broader and messier problem: it spans context management, output verification, incident governance, cross-session memory, and organizational discipline. OAgents does not claim the same protocol-level precision as OAuth. It claims the same structural role -- a trust layer that sits above the execution layer and makes delegation safe -- while acknowledging that the controls required are more complex and the standardization work is less mature.

That mechanism is the OAgent behavioral envelope.

1.3 Scope

This profile applies to AI agents operating in the following deployment context:

- Role: Operational agent with authority to take consequential actions (code deployment, infrastructure management, content generation, incident response)
- Autonomy level: Semi-autonomous to autonomous, with defined human oversight thresholds
- Environment: Enterprise operational environments with multiple interdependent systems
- Model: Any large language model (the standard is model-agnostic by design)

This profile does not address AI systems in the following contexts, which may require separate profiles: pure inference systems without operational authority, consumer-facing conversational AI without consequential action capabilities, or embedded AI in physical systems (robotics, autonomous vehicles).

This profile is scoped to operational reliability and behavioral trust. The AI RMF trustworthiness characteristics of Privacy-Enhanced and Fair with Harmful Bias Managed are out of scope for this profile and are not addressed by the behavioral envelope. Organizations deploying OAgents in contexts where these characteristics are material should pair this profile with companion frameworks identified in Appendix B.

1.4 Terminology

OAgent. An AI agent operating within a behavioral envelope that provides structured guarantees about its operational behavior. An OAgent is conformant with this standard when it implements the components specified at the applicable compliance level.

OAgents. The open standard defining behavioral envelope requirements, component taxonomy, and compliance levels for trustworthy AI agent operations.

Behavioral envelope. The structured set of pre-execution gates, post-execution verification, and operational discipline mechanisms that bound an OAgent's behavior.

Model-agnostic. Behavioral guarantees that function identically regardless of which large language model provides the underlying computation.

TEVV. Test, Evaluation, Verification, and Validation, as defined in AI RMF 1.0.

RFC 2119 normative language. MUST indicates a requirement; SHOULD indicates a recommendation; MAY indicates an option.

1.5 Intended Adopters

This standard is designed for three primary adopter audiences.

Infrastructure and platform engineering teams deploying AI agents to manage software systems, cloud infrastructure, CI/CD pipelines, or operational monitoring.

Internal AI platform groups building shared AI agent services for their organizations.

Vendors building AI agent platforms who need to demonstrate behavioral trustworthiness to enterprise customers.

This standard is not designed for -- and should not be applied to -- consumer-facing conversational AI without operational authority, pure inference pipelines, HR or credit decisioning systems, or physical systems where safety-critical real-time constraints require separate certification frameworks.

2. AI RMF Alignment Statement

The OAgents standard is designed as a comprehensive implementation of the NIST AI RMF 1.0 for the enterprise AI agent operational context. It addresses all four AI RMF functions:

GOVERN. OAgents behavioral envelope components instantiate governance structures as executable mechanisms: session protocols enforce operational discipline, impact level

classification enforces risk-proportionate decision-making, and asset registries maintain organizational accountability.

MAP. OAgents knowledge injection and context classification components operationalize the MAP function's requirement to establish and understand context before acting.

MEASURE. OAgents quality gate components instantiate the MEASURE function's requirement for independent evaluation of AI system outputs.

MANAGE. OAgents enforcement mechanisms, incident lifecycle tracking, and lessons-learned pipeline operationalize the MANAGE function's requirement for active risk response and continuous improvement.

The complete subcategory mapping is provided in Appendix A.

3. Design Principles

The OAgents standard is built on five principles derived from production operational experience and grounded in the AI RMF's trustworthiness characteristics.

3.1 The Model Is a Commodity (AI RMF: Valid and Reliable; Secure and Resilient)

Behavioral guarantees must be independent of the underlying model. An organization that encodes operational trust into model-specific prompts or vendor-specific tool configurations has created lock-in at the trust layer -- the most critical layer to keep portable. The OAgents standard requires that all behavioral envelope components function identically across model providers.

This is a claim about trust-layer portability, not model equivalence. The standard does not assert that all models perform identically or that model selection is inconsequential. It asserts that the behavioral envelope -- memory systems, enforcement gates, quality review agents, and operational logging -- operates at the infrastructure layer, outside the model's context window and independent of any model-specific API.

3.2 Enforcement Over Documentation (AI RMF: Accountable and Transparent)

Documented standards are advisory. Under context pressure, deadline urgency, or prompt complexity, AI agents routinely violate documented procedures. The OAgents standard requires that critical behavioral guarantees be implemented as executable gates -- mechanisms that block non-compliant actions rather than merely discouraging them.

3.3 Independent Verification (AI RMF: Accountable and Transparent; Valid and Reliable)

No system can reliably evaluate its own output. An AI agent that generates code and reviews that code in the same context is structurally incapable of independent quality assurance. The OAgents standard requires that quality verification be performed by a separate process with independent context -- ideally a different model instance from a different provider.

3.4 Persistent Learning (AI RMF: Accountable and Transparent; Safe)

AI agent sessions are ephemeral by default. Without explicit persistence mechanisms, every session starts from zero operational knowledge. The OAgents standard requires that operational lessons, behavioral corrections, and contextual knowledge persist across session boundaries and be loaded into every new session.

3.5 Self-Improving Reliability (AI RMF: Safe; Secure and Resilient)

A compliant behavioral envelope must become more reliable over time, not less. Operational failures must feed back into the envelope's enforcement mechanisms, creating a closed loop from failure detection to automated prevention.

4. Behavioral Envelope Architecture

The behavioral envelope interposes structured gates between the user's request and the agent's action, and between the agent's output and its delivery.

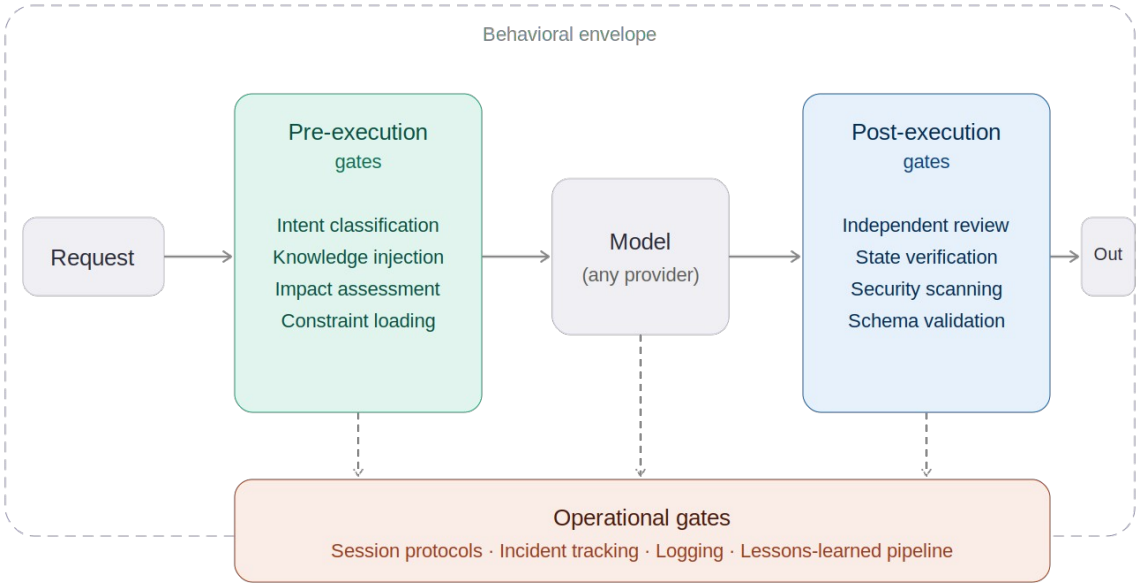

Figure 1. OAgents behavioral envelope architecture.
All agent actions pass through pre- and post-execution gates. Operational gates persist across sessions.

Figure 1. OAgents behavioral envelope architecture. All agent actions pass through pre- and post-execution gates. Operational gates persist across sessions. The model is replaceable; the envelope is not.

4.1 Pre-Execution Gates

Pre-execution gates shape context and constrain behavior before the model processes a request. They implement the AI RMF MAP function's requirement to establish context before acting.

Intent classification. Requests are categorized to determine which tools, constraints, and verification levels apply.

Knowledge injection. Persistent memory from prior sessions is loaded into the model's context.

Impact assessment. Each potential action is classified by blast radius on a defined scale (Level 1: read-only through Level 5: irreversible/external-facing).

Behavioral constraint loading. Domain-specific operational standards, quality requirements, and failure mode catalogs are loaded as active constraints.

4.2 Post-Execution Gates

Post-execution gates verify output quality and safety before delivery. They implement the AI RMF MEASURE function's TEVV requirements.

Independent quality review. A separate model instance reviews significant outputs against defined quality criteria.

State verification. Claims about system state are verified against reality using diagnostic commands before being asserted to the user.

Security scanning. Outputs are checked for credential exposure, injection vectors, and information leakage before delivery.

Schema validation. Structured outputs are validated against expected schemas before entering downstream systems.

4.3 Operational Gates

Operational gates impose discipline on the agent's lifecycle across sessions. They implement the AI RMF MANAGE function's requirements for ongoing risk management.

Session protocols. Mandatory startup and shutdown procedures bookend every operational session.

Incident lifecycle management. Operational failures are tracked through a structured lifecycle with cross-session persistence and recurrence detection.

Operational logging. Every significant action is logged as structured data with timestamps and correlation identifiers.

Lessons-learned feedback. Operational failures flow through a multi-stage pipeline: detection, lesson capture, quality criteria update, and enforcement gate installation.

When multiple OAgents operate in coordination, additional principles govern envelope interactions, incident ownership, shared memory access, and enforcement gate precedence. These coordination principles are specified in Section 12.1.

5. Component Taxonomy

The OAgents standard defines 26 components across 7 categories. Normative language follows RFC 2119: MUST indicates a conformance requirement; SHOULD indicates a recommendation for production deployments; MAY indicates an optional capability. Each component is mapped to AI RMF 1.0 subcategories in Appendix A.

5.1 Behavioral Shaping

Components that modify the agent's default behavior toward operational reliability. Addresses AI RMF trustworthiness characteristics: Valid and Reliable; Accountable and Transparent.

Component	Level	Description	AI RMF Subcategory
Persistent feedback memory	MUST	Behavioral corrections persist across sessions as structured entries with the rule, rationale, and application guidance. Loaded into every new session context.	GOVERN 1.2; GOVERN 4.1; MANAGE 4.1
Named failure mode catalog	MUST	Known agent failure patterns are explicitly named (over-confidence, stale references, hallucinated state, scope creep, instruction drift). The agent self-monitors for cataloged patterns.	MAP 2.2; MEASURE 2.5; MANAGE 1.3
Context degradation detection	SHOULD	The agent monitors its own reliability during long sessions, detecting increasing error rates or stale assumptions. Degradation triggers a recommendation to terminate the session.	MEASURE 2.6; MANAGE 1.1
Reasoning anchors	SHOULD	Stable epistemological commitments that ground reasoning under ambiguity: precision over vagueness, questioning assumptions, verification before assertion.	GOVERN 4.1; MAP 2.3

5.2 Quality Gates

Executable checkpoints that prevent defective work from advancing. Addresses AI RMF trustworthiness characteristics: Valid and Reliable; Accountable and Transparent; Safe.

Component	Level	Description	AI RMF Subcategory
Independent output review	MUST	Significant outputs are reviewed by a separate model instance with independent context. The reviewing model SHOULD be from a different provider. Results are logged with structured verdicts.	MEASURE 2.5; MEASURE 2.8; GOVERN 4.3
Process enforcement gate	MUST	Critical workflow steps are enforced by executable gates that block non-compliant actions. Advisory documentation alone is insufficient for MUST-level guarantees.	GOVERN 1.4; MANAGE 1.2; MANAGE 2.2
Security audit	SHOULD	Automated security checks run against outputs and affected systems before significant actions. Check categories include credential exposure, injection vectors, and access control.	MEASURE 2.7; MANAGE 1.3; GOVERN 6.1
Schema validation	SHOULD	Structured outputs are validated against defined schemas before acceptance into downstream systems.	MEASURE 2.5; MEASURE 1.1

5.3 Operational Discipline

Component	Level	Description	AI RMF Subcategory
Session lifecycle protocol	MUST	Mandatory start and wrap procedures verify system state, load context, and ensure clean handoffs. Protocol completion is verified programmatically.	GOVERN 1.5; MANAGE 4.1; GOVERN 2.1
Impact level classification	MUST	Actions are classified by blast radius (Level 1: read-only through Level 5: irreversible/external-facing). Higher levels require proportionally more verification.	MAP 3.5; MANAGE 1.1; GOVERN 3.2
Incident lifecycle tracking	SHOULD	Operational failures are tracked through a persistent lifecycle with cross-session continuity, recurrence detection, and severity escalation.	MANAGE 3.2; MEASURE 3.1; MANAGE 2.4
Structured operational logging	SHOULD	Every significant action is logged as structured data with timestamps and correlation identifiers. Logs are append-only and support cross-session pattern analysis.	GOVERN 1.4; MEASURE 3.1; MANAGE 4.2

5.4 Knowledge Injection

Component	Level	Description	AI RMF Subcategory
Persistent memory system	MUST	Typed memory entries persist across sessions. Entries include staleness metadata and MUST be verified against current state before being acted upon.	GOVERN 1.2; MAP 1.1; MANAGE 4.1
Domain skill loading	MUST	Operational knowledge (procedures, decision frameworks, service-specific details) is loaded as active context for every session.	GOVERN 1.2; MAP 1.1; MAP 3.4
Lessons-learned pipeline	SHOULD	Operational failures flow through a closed-loop pipeline: detection, memory capture, quality criteria update, and enforcement installation.	MANAGE 4.2; GOVERN 5.2; MEASURE 4.1

5.5 Enforcement Mechanisms

Component	Level	Description	AI RMF Subcategory
Executable action gates	MUST	Critical behavioral guarantees are implemented as gates that block non-compliant actions at the tooling level (e.g., version control hooks, CI/CD checks).	GOVERN 1.4; MANAGE 2.2; GOVERN 4.3
Severity escalation rules	SHOULD	Recurrence of operational failures automatically escalates severity. Third occurrence generates a critical alert and structural review.	MANAGE 1.3; MANAGE 3.2; MEASURE 3.2
Protocol compliance verification	SHOULD	Session protocol completion is verified programmatically at session wrap. Non-completion blocks session closure until resolved.	GOVERN 1.5; MANAGE 2.2; GOVERN 2.1

5.6 Project Governance

Component	Level	Description	AI RMF Subcategory
Centralized work tracking	SHOULD	A single backlog spanning all managed systems, with structured priority and status fields. Reviewed and updated during every session wrap.	GOVERN 1.6; MANAGE 4.2; GOVERN 2.1
Platform sovereignty	SHOULD	Critical operational data exists on at least two independent platforms simultaneously. No single vendor removal can cause data loss.	GOVERN 6.2; MANAGE 1.4; MAP 4.2
Asset registry	SHOULD	A maintained inventory of all managed systems, accounts, and platforms with sync status and gap identification.	GOVERN 1.6; MAP 4.1; MAP 4.2
Vendor independence	SHOULD	Product-critical automation lives in version-controlled code, not in third-party workflow tools.	GOVERN 6.1; GOVERN 6.2; MAP 4.1

5.7 Anti-Hallucination

Addresses NIST AI 600-1 GenAI risk: Hallucination.

Component	Level	Description	AI RMF Subcategory
State verification protocol	MUST	Before asserting any claim about system state, the agent MUST verify against current reality using appropriate diagnostic methods. Memory of a state is not confirmation of a state.	MEASURE 2.5; MAP 2.2; GOVERN 4.2
Memory staleness detection	MUST	Memory entries are treated as claims about past state. Before acting on a memory, the agent verifies it against current system state. Conflicts are resolved in favor of observed reality.	MEASURE 2.6; MAP 1.1; MANAGE 1.3
Hallucination tracking	SHOULD	Detected hallucinations are logged with the false claim, the truth, and the detection method. Recurrence patterns trigger quality criteria updates.	MEASURE 3.1; MEASURE 4.1; MANAGE 4.2

6. Conformance and Certification

6.1 How Conformance Is Established

OAgents conformance is established through evidence -- observable artifacts demonstrating that the specified components are implemented and operational. Conformance is not established by assertion. The complete set of evidence criteria for MUST-level components is specified in Appendix C.

6.2 Conformance Verification by Level

Level 1 (OAgent-Basic) -- Self-Assessment. Conformance is verified by the deploying organization against the Appendix C checklist.

Level 2 (OAgent-Standard) -- Documented Evidence Review. Conformance is verified through a structured evidence review by an internal audit team or external reviewer.

Level 3 (OAgent-Autonomous) -- Third-Party Verification. Conformance is verified by an independent third-party assessor. The third-party assessor model is analogous to FedRAMP's Third Party Assessment Organization (3PAO) structure.

6.3 Certification Status

Formal certification programs for OAgents conformance are not yet established. Level 2 conformance criteria are designed with compatibility with FedRAMP Moderate control requirements in mind. A formal FedRAMP alignment mapping is planned as a future deliverable.

7. Compliance Levels

The OAgents standard defines three compliance levels to support incremental adoption consistent with the AI RMF's design as a flexible, voluntary framework.

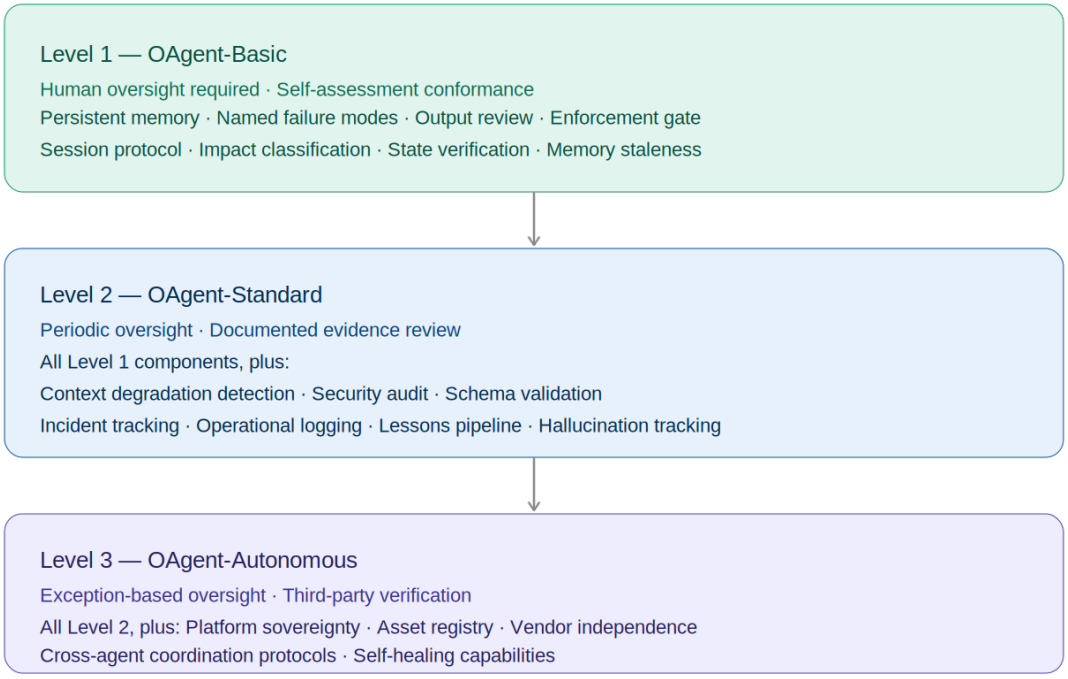

Figure 2. OAgents compliance levels. Each level adds components and increases conformance verification rigor.

Figure 2. OAgents compliance levels. Each level adds components and increases conformance verification rigor.

Level 1: OAgent-Basic

All MUST components are implemented. Suitable for supervised operational use with human oversight.

- Persistent feedback memory
- Named failure mode catalog
- Independent output review
- At least one process enforcement gate
- Session lifecycle protocol (startup and shutdown verification)
- Impact level classification for all actions
- Persistent memory system with staleness detection
- Domain skill loading
- At least one executable action gate
- State verification before all assertions
- Memory staleness detection

Trustworthiness assurance: Level 1 provides baseline guarantees of valid and reliable behavior, minimal accountability structures, and anti-hallucination protections.

Level 2: OAgent-Standard

All MUST and SHOULD components are implemented. Suitable for production use with periodic human oversight. Level 1 plus: Context degradation detection, Reasoning anchors, Security auditing, Schema validation, Incident lifecycle tracking, Structured operational logging, Lessons-learned pipeline, Severity escalation rules, Protocol compliance verification, Centralized work tracking, Hallucination tracking.

Trustworthiness assurance: Level 2 provides comprehensive operational guarantees across all seven AI RMF trustworthiness characteristics.

Level 3: OAgent-Autonomous

Level 2 compliance plus: Platform sovereignty, Asset registry maintenance, Vendor independence verification, Cross-agent coordination protocols consistent with the principles specified in Section 12.1 (full formal specification is future work), Self-healing capabilities.

Trustworthiness assurance: Level 3 provides guarantees suitable for high-autonomy deployments with minimal human oversight.

8. Compounding Effects

Individual components provide linear value. The OAgents standard's architecture creates five compounding effects in which component interactions produce multiplicative reliability gains over time.

8.1 Lesson-to-Enforcement Pipeline (AI RMF: MANAGE 4.2; GOVERN 5.2)

Failure detection feeds lesson capture, which updates quality criteria, which installs enforcement gates. Four components create a closed loop where every failure permanently strengthens the envelope.

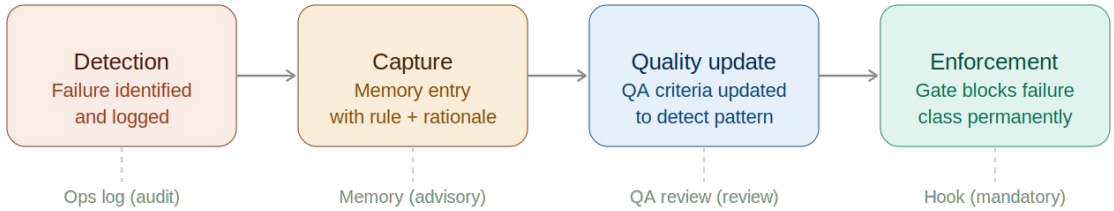

Figure 3. Lesson-to-enforcement pipeline. Every operational failure produces four layers of defense against recurrence. The system strengthens from its failures rather than repeating them.

Figure 3. Lesson-to-enforcement pipeline. Every operational failure produces four layers of defense against recurrence. The system strengthens from its failures rather than repeating them.

8.2 Advisory Plus Mandatory (AI RMF: GOVERN 1.4; MANAGE 2.2)

Behavioral memory shapes agent behavior (advisory). Executable gates block violations (mandatory). Advisory systems degrade under context pressure. Mandatory systems do not. The combination is strictly stronger than either mechanism alone.

8.3 Cross-Session Continuity (AI RMF: MANAGE 4.1; GOVERN 1.5)

Persistent memory, session protocols, and operational logging together create continuous operational awareness across inherently ephemeral sessions.

8.4 Gap-to-Governance (AI RMF: GOVERN 5.2; MANAGE 4.2)

When operational gaps are discovered, the lessons-learned pipeline installs structural fixes within the same session. This operationalizes GOVERN 5.2.

8.5 Sovereignty by Default (AI RMF: GOVERN 6.1; GOVERN 6.2)

Platform sovereignty and vendor independence together create a system that is inherently portable. No single vendor dependency exists for critical operational state.

9. Reference Implementation

The OAgents behavioral envelope has been implemented as a reference system operating in a development laboratory context. The reference implementation runs on a multi-host Linux environment and manages a suite of containerized services including authentication, collaboration, communication, and version control systems. This is a single-organization development environment, not a scaled production deployment. The AI operator is a large language model accessed via API, with a separate model instance from a different provider performing independent quality review.

A sanitized snapshot of the reference implementation is published at https://github.com/ologos-corp/OAgents-standard/tree/main/reference. This snapshot constitutes the evidence artifacts that Appendix C requires conforming implementations to produce.

Human-Curated, AI-Enabled (HCAE) methodology. The reference implementation was developed and operated using an HCAE methodology: a human architect defined operational standards, curated behavioral corrections, and determined conformance expectations; an AI operator implemented and maintained the behavioral envelope under those standards. This is not incidental to the standard’s design -- it is the operational model the standard is built to support and constrain.

9.1 Reference Stack Summary

Component Category	Implementation Approach	Primary Artifact	Reference File
Behavioral Shaping	YAML-frontmatter memory entries with typed fields	Structured memory store + session startup load log	memory/example_feedback.md, scripts/session-start.py
Quality Gates	Separate model instance (different provider) via API; structured JSON verdict logged per review	QA review log with model IDs and verdicts	qa-agent/agent.py, ops/examples/qa_log.json
Operational Discipline	Python session scripts enforcing mandatory start/wrap procedures	Session startup and wrap logs with compliance status	scripts/session-start.py, scripts/session-wrap.py
Knowledge Injection	Markdown skill files and memory index loaded at session start	Skill files + memory index with load confirmation	skills/sysadmin.md, memory/MEMORY.md
Enforcement Mechanisms	Git pre-commit hook blocks commit without QA review; pre-push hook enforces dev-first workflow	Hook source code (version-controlled) + activation log	hooks/pre-commit, hooks/pre-push
Project Governance	Structured JSON ops logs (append-only); centralized backlog in GitHub Projects	Ops log files with schema definitions	ops/README.md, ops/examples/sysadmin_log.json
Anti-Hallucination	State verification commands before assertions; memory staleness check on load	Verification commands in scripts; QA log entries flagging false assertions	skills/quality_assurance.md, qa-agent/prompts/qa_review.md

9.2 Operational Metrics

The following metrics are drawn from live operational logs of the reference implementation. The reference implementation operates as a development laboratory environment. Structured logging has been active since 2026-04-02; the system has been in continuous operation since approximately mid-March 2026.

Metric	Count	Source
Git commits (each triggers pre-commit and pre-push hook enforcement)	367	git log
Independent QA reviews executed	83	ops/qa_log.json
QA pass rate	97.6% (81 pass, 2 fail)	ops/qa_log.json
Automated health check executions	349	ops/ sysadmin_log.json
Security audit runs	4	ops/security_log.json
Lessons learned captured (lesson-to-enforcement pipeline)	9	ops/ lessons_learned.json
Behavioral correction memories accumulated	23	memory/ feedback_*.md
Total persistent memory entries	49	memory/*.md
Hallucination catches logged	2	ops/qa_log.json

These figures demonstrate operational use, not empirical benchmarking. The 367 commits with associated hook enforcement events show the enforcement gate has been active across the full project lifetime. The 83 independent QA reviews -- each performed by a model instance from a different provider -- show the independent review mechanism has been exercised at meaningful volume. No repeat occurrence of the same failure pattern has been observed after enforcement installation -- the pipeline's primary design intent.

9.3 Scope and Limitations

The reference implementation demonstrates architectural feasibility of all 26 components. It is not offered as empirical validation of the standard's effectiveness claims. Systematic empirical validation is identified as community work. The appropriate sequence for a v1.0 specification is: define the taxonomy, demonstrate feasibility, invite community validation.

10. Related Work

10.1 Agent Frameworks (LangChain, CrewAI, AutoGen, MetaGPT)

These frameworks address agent orchestration. Behavioral trustworthiness during execution is outside their scope. OAgents addresses the trust layer that sits above any agent framework.

10.2 Workflow Automation (n8n, Zapier, Apache Airflow)

These platforms provide workflow orchestration for structured, repeatable processes. OAgents addresses the behavioral governance of judgment-exercising agents -- a problem that workflow orchestration does not encounter.

10.3 AI Development Environments (GitHub Copilot, Cursor, Claude Code)

These tools are designed for interactive developer assistance. OAgents addresses autonomous or semi-autonomous operational agents with persistent memory, cross-session accountability, and enforcement-backed behavioral constraints.

10.4 Output Validation Frameworks (Guardrails AI, NeMo Guardrails)

These frameworks provide output validation corresponding to the post-execution gate layer of OAgents. OAgents encompasses output validation as one component within a broader framework additionally addressing pre-execution shaping, cross-session memory, operational discipline, and enforcement mechanisms.

10.5 Agent Behavioral Contracts (ABC)

Bhardwaj (2026) introduces Agent Behavioral Contracts, a formal framework applying Design-by-Contract principles to AI agents with probabilistic compliance frameworks and mathematical drift bounds. The two approaches address complementary concerns: ABC focuses on formal runtime specification within a session; OAgents focuses on operational completeness across sessions. The frameworks could be composed.

10.6 Organizational AI Management Standards (ISO/IEC 42001)

ISO/IEC 42001 operates at the organizational management system level. OAgents operates at the individual agent level. These are complementary scopes.

10.7 Regulatory and Standards Frameworks

NIST AI RMF 1.0 (NIST AI 100-1, 2023). The foundational governance framework this profile extends. OAgents is not a competitor to the AI RMF -- it is an implementation profile of it.

NIST AI 600-1 -- Generative AI Profile (2024). OAgents explicitly addresses hallucination through the Anti-Hallucination component category. Full mapping to all 13 risk categories is planned for a subsequent revision.

Executive Order 14110 (2023). OAgents is designed to be consistent with and supportive of EO 14110 implementation requirements.

NIST SP 800-207 -- Zero Trust Architecture (2020). SP 800-207 provides the conceptual precedent: trust must be verified continuously through explicit mechanisms, not assumed.

11. Adoption Path

The OAgents standard is designed for incremental adoption with a parallel track through the standards body process.

Phase 1: Specification (Current). Publish the component taxonomy, compliance levels, and AI RMF mapping as an open specification. Submit to NIST AI RMF community as an Implementation Profile contribution. Engage with NIST's AI Agent Standards Initiative (CAISI, 2026).

Phase 2: Reference Implementations. Open-source reference implementations for popular agent frameworks. Public comment period on v1.0 specification. Publish v1.1.

Phase 3: Tooling and Federal Alignment. Development tools for OAgent-compliant systems. Formal FedRAMP alignment mapping. Engage IEEE and OASIS for formal standardization.

Phase 4: Ecosystem and Certification. Independent quality review services. Certification program modeled on FedRAMP's 3PAO structure. Standards lifecycle: v1.0 → v1.1 → v2.0.

12. Limitations and Open Problems

Context window cost. The behavioral envelope consumes model context capacity. Risk-proportional context loading is the recommended mitigation.

Bootstrap period. A new OAgent deployment starts with an empty behavioral envelope. Pre-built behavioral templates by domain can reduce bootstrap time.

Verification latency. Independent quality review and state verification add latency. Risk-proportional verification is the recommended mitigation.

Empirical validation. Systematic empirical measurement remains to be conducted across diverse deployment contexts by the community.

Multi-agent coordination. The current specification is fully defined for single-agent deployments. Coordination principles for multi-agent environments are described in Section 12.1.

Formal verification. Formal verification of behavioral envelope properties would strengthen the standard's guarantees and is identified as future work.

NIST AI 600-1 full coverage. Full mapping to all 13 GenAI risk categories is planned for a subsequent revision.

12.1 OAgents in Multi-Agent Environments

The following principles govern OAgents coordination in multi-agent environments, pending full formalization in a subsequent revision.

Each OAgent maintains its own envelope. A compliant OAgent does not inherit behavioral guarantees from the orchestrating agent. An orchestrated OAgent MUST apply its own pre-execution gates to instructions received from another agent.

Incident ownership follows action authority. The agent that takes a consequential action is the incident owner. The orchestrator SHOULD maintain a cross-agent incident summary.

Independent quality review applies at output boundaries. A compliant implementation SHOULD include at least one independent review gate at each significant inter-agent handoff.

Shared memory requires versioned access. Memory entries MUST include an authoring agent identifier and timestamp. Agents MUST NOT overwrite entries authored by other agents without explicit conflict resolution.

Enforcement gates are non-negotiable across agents. An orchestrating agent cannot instruct a subordinate OAgent to bypass its enforcement gates.

13. Conclusion

Enterprise AI agents operating in high-autonomy roles -- deploying code, managing infrastructure, responding to incidents -- face a trust gap that model capability cannot close. The standards infrastructure to address this gap is nascent: identity delegation has OAuth, network trust has SP 800-207, but a control and conformance profile for operational AI agent behavioral trust has no widely adopted equivalent. OAgents proposes a contribution to that developing landscape -- a behavioral envelope standard that makes AI agent operations reliable, auditable, and self-improving for enterprise operational contexts.

The standard is presented as an AI RMF Implementation Profile, grounding its 26 components in the NIST AI RMF 1.0 functions, categories, and subcategories. The taxonomy is open. No organization needs permission to implement OAgents. Any organization that implements the MUST-level components at the specified compliance level is conformant.

We invite NIST, IEEE, OASIS, and the broader standards community to evaluate this taxonomy, contribute replication evidence, propose revisions, and engage in the public comment process as the standard matures. We specifically invite federal agencies operating under EO 14110 to evaluate OAgents as an implementation framework for the AI agent trustworthiness requirements the Executive Order mandates.

The behavioral envelope is to operational AI agents what OAuth was to identity delegation -- not a product, but a trust layer that makes delegation safe. For high-autonomy agents in consequential enterprise roles, that layer is what responsible deployment requires. The model is a commodity. Trust is the product.

References

[1] National Institute of Standards and Technology. (2023). Artificial Intelligence Risk Management Framework (AI RMF 1.0) (NIST AI 100-1). https://doi.org/10.6028/NIST.AI.100-1

[2] National Institute of Standards and Technology. (2024). AI Risk Management Framework: Generative Artificial Intelligence Profile (NIST AI 600-1). https://doi.org/10.6028/NIST.AI.600-1

[3] Hardt, D. (Ed.). (2012). The OAuth 2.0 Authorization Framework (RFC 6749). IETF. https://doi.org/10.17487/RFC6749

[4] Brown, T. B., et al. (2020). Language models are few-shot learners. Advances in Neural Information Processing Systems, 33.

[5] Schick, T., et al. (2023). Toolformer: Language models can teach themselves to use tools. Advances in Neural Information Processing Systems, 36.

[6] Hong, S., et al. (2024). MetaGPT: Meta programming for a multi-agent collaborative framework. ICLR.

[7] Shinn, N., et al. (2023). Reflexion: Language agents with verbal reinforcement learning. Advances in Neural Information Processing Systems, 36.

[8] Yao, S., et al. (2023). ReAct: Synergizing reasoning and acting in language models. ICLR.

[9] Bhardwaj, V. P. (2026). Agent behavioral contracts. arXiv:2602.22302.

[10] National Institute of Standards and Technology. (2026). AI Agent Standards Initiative. CAISI. https://www.nist.gov/caisi/ai-agent-standards-initiative

[11] Casper, S., et al. (2026). 2025 AI Agent Index. arXiv:2602.17753.

[12] National Institute of Standards and Technology. (2020). NIST Privacy Framework v1.0. https://doi.org/10.6028/NIST.CSWP.01162020

[13] National Institute of Standards and Technology. (2022). Towards a Standard for Identifying and Managing Bias in AI (NIST SP 1270). https://doi.org/10.6028/NIST.SP.1270

[14] ISO/IEC 23894:2023. Artificial intelligence -- Guidance on risk management. ISO.

[15] Executive Office of the President. (2023). Executive Order 14110 on Safe, Secure, and Trustworthy AI. 88 FR 75191.

[16] National Institute of Standards and Technology. (2020). Zero Trust Architecture (NIST SP 800-207). https://doi.org/10.6028/NIST.SP.800-207

Corresponding author: JD Longmire, Northrop Grumman Fellow (unaffiliated research). jdlongmire@outlook.com. ORCID: 0009-0009-1383-7698

All components described in this standard are implemented in the reference system. No generalized empirical benchmarks are claimed.

This document is submitted as a community contribution to the NIST AI RMF. Comments and replication results may be directed to the author.